

Hardware-Software Co-Design for Embedded Systems

Software based functionality in today's embedded products causes delays in project completion if you wait for the hardware prototype to begin software development and debugging. Concurrent design and verification of hardware and software is a trend that reduces time-to-market

■ SHWETA DHADIWAL BAID

One of the most time-consuming processes in embedded systems development is hardware-software integration and redesign, which is iterative until you meet the end requirements. Waiting for the hardware prototype to begin software debugging is no more a viable option with demanding time-to-market and the cost involved in redesign. Many tool vendors are now pushing the co-design and co-verification approach to address this issue.

Traditionally, after collecting the system requirements, the design team identifies the requirements that should be met using hardware blocks and the software blocks, and then starts sequential development. "The traditional disaggregated development approach, where hardware is developed first and software later, constrains the innovation that today's technology makes possible," says Rahul Arya, director, marketing & technology sales, Cadence Design Systems.

"If the goal is an optimal design in terms of cost, performance, time-to-market and power consumption, the design process must thoroughly consider both hardware and software aspects together early on," explains Ashish Gupta, founding member and R&D engineer at The Learning Labs. "Hardware/software co-design approach will help you to avoid shocks and surprises at the implementation/validation stage," he adds. Co-verification ensures

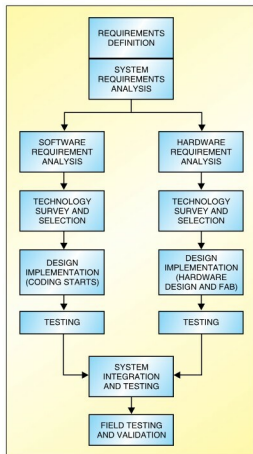


Fig. 1: Traditional design flow (Courtesy: Ashish Gupta, The Learning Labs)

that the system meets the initial set of requirements through its hardware and software blocks put together.

Hardware-software partitioning

Embedded systems for real-time applications are implemented as a mix of hardware and software sub-systems

interacting in one cohesive environment.

Moorthy Palanisamy, managing director, Oxys Technologies, explains, "The hardware part is mainly tuned for performance, while the software part implements most of the product's features and provides flexibility to the product for customisation and easy changes. Many a times designers strive to make everything fit in software and off-load only some parts of the design to hardware in order to meet timing constraints."

"Hardware/software partitioning is all about deciding which scheme gives you the best trade-off with respect to the known requirement metrics," explains Gupta. Justifying this with an example, he says, "In a simple music player design, you can implement the decoding algorithm for compressed music files using software and a high-speed processor. Else, you can opt for an off-the-shelf ASIC that

is capable of MP3 decoding. If the first choice makes you write lengthy code for decompression algorithms, the latter will restrict upgrading the product to support multiple compression formats."

Indicating the challenges in independent hardware and software design approach, Moorthy shares, "Separate